Assignment III: Mini-RISC CPU with Data Memory, Load/Store, and Pipeline

Department of Computer Science and Engineering

Objective

In Assignment I you built an ISA-level simulator. In Assignment II you built a micro-architectural CPU with datapath and control signals. In this assignment you will extend the CPU into a more realistic processor by adding:

- Data Memory
- Load/Store Instructions
- A 2-stage Pipeline (IF, EX/WB)
- Hazard detection and stalling
- Performance counters (cycles, CPI)

New Components

- Data Memory: 256 × 8-bit
- MAR, MDR
- Pipeline Register: IF/EX

New Instructions

Opcode	Mnemonic	Operation
0xD	LD Rn, addr	Rn ← MEM[addr]
0xE	ST Rn, addr	MEM[addr] ← Rn

Architecture Rule

- Only LD and ST can access memory
- ALU cannot access memory directly
- This is a load/store architecture

Detailed Explanation of the 2-Stage Pipeline

This processor uses a very simple pipeline with two stages:

```
Stage 1: IF  (Instruction Fetch)
Stage 2: EX  (Execute + Memory + Writeback)
```

In every clock cycle, the processor attempts to do the following in parallel:

- The IF stage fetches a new instruction from Instruction Memory.
- The EX stage executes the previously fetched instruction.

A pipeline register `IF/EX` sits between these two stages and stores the instruction fetched in the IF stage so that it can be executed in the next cycle.

Pipeline Timing Example (No Hazards)

For three instructions I_1, I_2, I_3:

```
Cycle 1: IF(I1)
Cycle 2: EX(I1) | IF(I2)
Cycle 3: EX(I2) | IF(I3)
Cycle 4: EX(I3)
```

Thus, after the pipeline is full, ideally one instruction completes every cycle.

What is a Stall (Bubble)?

A **stall** (also called a **bubble**) is a cycle in which the pipeline is **intentionally stopped** to avoid using incorrect data.

When a stall is inserted:

- The EX stage is allowed to finish.
- The IF stage is **frozen** (PC is not incremented).
- No new instruction is fetched.

What is a Flush?

A **flush** means **discarding a wrong instruction** that has already been fetched.

This happens in case of **control hazards** (jump/branch instructions).

If a jump is taken:

- The instruction that was fetched speculatively is **invalid**.
- The pipeline register is cleared.
- The next instruction is fetched from the correct target address.

Data Hazard Model Used in This Assignment

In this assignment, you only need to detect and handle **Load-Use Data Hazards**.

What is a Load-Use Hazard?

```
LD R1, 10
ADD R1
```

The `LD` instruction produces the value of `R1` only at the **end of the EX stage**. The next instruction `ADD R1` needs `R1` at the **beginning of the EX stage**.

Therefore, without stalling, the `ADD` instruction will read an **old incorrect value**.

Rule Used

> If instruction in EX stage is `LD Rk, addr` and instruction in IF stage uses register `Rk`, then insert exactly **one stall cycle**.

Cycle-by-Cycle Example With a Stall

For the program:

```
LD R1, 10
ADD R1
HALT
```

Execution timeline:

```
Cycle 1: IF(LD)
Cycle 2: EX(LD) | IF(ADD)
Cycle 3: EX(LD finishes) | STALL (ADD waits)
Cycle 4: EX(ADD)
Cycle 5: EX(HALT)
```

How Load and Store Instructions Work

Load

```
LD R1, addr
```

- Read DataMemory[addr]
- Write the value into R1

Store

```
ST R1, addr
```

- Read value from R1
- Write it into DataMemory[addr]

What You Should Observe During Simulation

- Cycles where no new instruction is fetched (stall cycles)
- Flushed instructions after jumps
- Total cycles, total instructions, CPI
- Programs with hazards take more cycles than programs without hazards

C Skeleton Code (Pipeline + Memory + Hazards)

```c
/* ============================================================
   Mini-RISC CPU     Assignment III
   Data Memory + Load/Store + 2-stage Pipeline + Hazards
   ============================================================ */

#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MEM_SIZE 256

typedef struct {
    unsigned char R[4];
    unsigned char ACC;
    unsigned char PC;
    unsigned char IR;
    unsigned char Z, C;
} CPU;

unsigned char IMEM[MEM_SIZE];
unsigned char DMEM[MEM_SIZE];

typedef struct {
    unsigned char IR;
    int valid;
} IFEX;

CPU cpu;
IFEX ifex;

int cycles = 0;
int instructions = 0;
int stalls = 0;

// Utility
void reset_cpu() {
    memset(&cpu, 0, sizeof(CPU));
    memset(&ifex, 0, sizeof(IFEX));
}

void load_program(const char *fname) {
    FILE *fp = fopen(fname, "r");
    if (!fp) { printf("Cannot open program file!\n"); exit(1); }
    int addr = 0; unsigned int x;
    while (fscanf(fp, "%x", &x) != EOF) IMEM[addr++] = (unsigned
       char)x;
    fclose(fp);
}

```

```
49 // Helpers
50 int is_load(unsigned char ir) { return (((ir >> 4) & 0xF) == 0xD)
   ; }
51
52 int uses_register(unsigned char ir, int reg) {
53     unsigned char op = (ir >> 4) & 0xF;
54     unsigned char operand = ir & 0xF;
55     switch (op) {
56         case 0x1: case 0x4: case 0x5: case 0x6:
57         case 0x7: case 0x8: case 0x9:
58             return (operand == reg);
59         default: return 0;
60     }
61 }
62
63 int detect_data_hazard(unsigned char ex_ir, unsigned char if_ir)
   {
64     if (!is_load(ex_ir)) return 0;
65     int loaded_reg = ex_ir & 0xF;
66     if (uses_register(if_ir, loaded_reg)) return 1;
67     return 0;
68 }
69
70 // ALU
71 unsigned char alu(unsigned char a, unsigned char b, int op) {
72     unsigned int res = 0;
73     switch (op) {
74         case 0x4: res = a + b; break;
75         case 0x5: res = a - b; break;
76         case 0x6: res = a * b; break;
77         case 0x7: res = (b == 0) ? 0 : a / b; break;
78         default: res = a;
79     }
80     cpu.Z = (res == 0);
81     cpu.C = (res > 255);
82     return (unsigned char)(res & 0xFF);
83 }
84
85 int jump_taken = 0;
86
87 void execute_stage() {
88     if (!ifex.valid) return;
89
90     unsigned char ir = ifex.IR;
91     unsigned char op = (ir >> 4) & 0xF;
92     unsigned char operand = ir & 0xF;
93
94     instructions++;
95
96     switch (op) {
97         case 0x1: cpu.ACC = cpu.R[operand]; break;
```

```
98          case 0x2: cpu.R[operand] = cpu.ACC; break;
99          case 0x3: cpu.ACC = operand; break;
100         case 0x4: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x4);
                break;
101         case 0x5: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x5);
                break;
102         case 0x6: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x6);
                break;
103         case 0x7: cpu.ACC = alu(cpu.ACC, cpu.R[operand], 0x7);
                break;
104         case 0xD: cpu.R[operand] = DMEM[operand]; break;
105         case 0xE: DMEM[operand] = cpu.R[operand]; break;
106         case 0xA: cpu.PC = operand; jump_taken = 1; break;
107         case 0xF: printf("HALT\n"); exit(0);
108     }
109
110     ifex.valid = 0;
111 }
112
113 void fetch_stage() {
114     ifex.IR = IMEM[cpu.PC++];
115     ifex.valid = 1;
116 }
117
118 int main() {
119     reset_cpu();
120
121     DMEM[0] = 5;
122     DMEM[1] = 3;
123
124     load_program("program.hex");
125
126     unsigned char next_ir;
127
128     while (1) {
129         cycles++;
130         jump_taken = 0;
131
132         int stall = 0;
133         if (ifex.valid) {
134             next_ir = IMEM[cpu.PC];
135             if (detect_data_hazard(ifex.IR, next_ir)) {
136                 stall = 1;
137                 stalls++;
138             }
139         }
140
141         execute_stage();
142
143         if (jump_taken) {
144             ifex.valid = 0;
```

```
145                 continue;
146             }
147 
148             if (!stall) {
149                 fetch_stage();
150             }
151         }
152         return 0;
153 }
```

Sample Test Programs

Test 1: Basic Load + Add + Store

```
LD R0, 0
LD R1, 1
LOAD R0
ADD R1
STORE R2
ST R2, 2
HALT
```

Test 2: Data Hazard (Must Stall)

```
LD R1, 0
ADD R1
HALT
```

Test 3: Chain Dependency

```
LD R1, 0
ADD R1
ADD R1
ADD R1
HALT
```

Test 4: Store/Load

```
MOVI 7
STORE R0
ST R0, 10
LD R1, 10
ADD R1
HALT
```

Test 5: Branch Flush

```
MOVI 0
CMP R0
JZ 10
MOVI 5   ; must be flushed
```

Submission

- Source code
- At least 5 program.hex files
- Execution logs

- Short report (3–5 pages)

Learning Outcome

Students will understand memory systems, load/store architectures, pipelining, hazards, stalls, flushes, and CPI.